

Введение в нейронные сети

От математической модели нейрона до современных архитектур

DS Summer 2025

Содержание

- 1 Математическая модель нейрона
- 2 Эволюция нейронных сетей
- 3 Современные архитектуры и типы нейронных сетей
- 4 Функции активации
- 5 Основные компоненты нейронных сетей
- 6 Ликбез по методам оптимизации
- 7 Обучения нейронных сетей
- 8 Для каких задач подходят нейронные сети, а для каких ML методы

Биологический нейрон vs искусственный нейрон

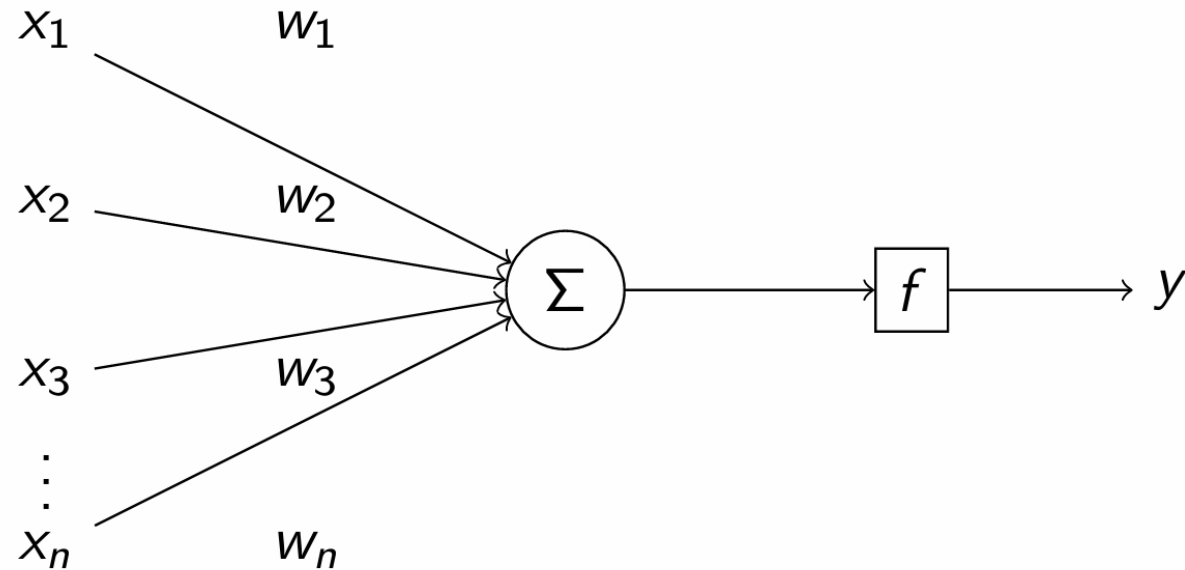
Биологический нейрон:

- Дендриты (входы)
- Ядро (обработка)
- Аксон (выход)
- Синапсы (веса)

Искусственный нейрон:

- Входные сигналы $x_1, x_2, \dots, x_n$
- Веса $w_1, w_2, \dots, w_n$
- Функция активации f
- Выходной сигнал y

Математическая модель перцептрона



Математическая формула:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right) = f(w^T x + b)$$

где b — смещение (bias)

История развития нейронных сетей (1943-1980)

- **1943** — Маккалок и Питтс: первая математическая модель нейрона
- **1949** — Дональд Хебб: правило обучения Хебба
- **1957** — Фрэнк Розенблатт: перцептрон
- **1969** — Минский и Пейперт: ограничения перцептрона (XOR проблема)
- **1974-1982** — Развитие метода обратного распространения ошибки

Первая зима ИИ (1970-1980): снижение интереса к нейронным сетям из-за ограничений перцептрона

Современная эра (1980-настоящее время)

- **1986** — Румельхарт, Хинтон, Уильямс: алгоритм backpropagation
- **1989** — Лекун: сверточные нейронные сети (CNN)
- **1997** — Хохрайтер и Шмидхубер: LSTM
- **2006** — Хинтон: глубокое обучение (deep learning)
- **2012** — AlexNet: прорыв в computer vision
- **2017** — Transformer: революция в NLP
- **2018-2024** — Эра больших языковых моделей (GPT, BERT, ChatGPT)

Ключевые факторы успеха: увеличение вычислительной мощности, большие данные, улучшенные алгоритмы

Основные типы нейронных сетей

1 Feedforward Neural Networks (FNN)

- Многослойный перцептрон (MLP)
- Полносвязные слои

2 Convolutional Neural Networks (CNN)

- Сверточные слои
- Компьютерное зрение

3 Recurrent Neural Networks (RNN)

- LSTM, GRU
- Обработка последовательностей

Специализированные архитектуры

- **ResNet** — остаточные связи для очень глубоких сетей
- **DenseNet** — плотные соединения между слоями
- **U-Net** — архитектура для сегментации изображений
- **Vision Transformer (ViT)** — применение transformer к изображениям
- **Graph Neural Networks (GNN)** — работа с графовыми данными
- **Neural ODE** — нейронные обыкновенные дифференциальные уравнения

Тенденция: переход от ручного проектирования архитектур к автоматическому поиску (Neural Architecture Search)

Классические функции активации

Сигмоида:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Выход: $(0, 1)$
- Проблема: затухание градиента

Гиперболический тангенс:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Выход: $(-1, 1)$
- Лучше чем сигмоида

ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

- Простая и эффективная
- Проблема: "мертвые" нейроны

Leaky ReLU:

$$\text{LReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

- Решает проблему "мертвых" нейронов

Современные функции активации

- **ELU (Exponential Linear Unit):**

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

- **Swish:**

$$\text{Swish}(x) = x \cdot \sigma(x)$$

- **GELU (Gaussian Error Linear Unit):**

$$\text{GELU}(x) = x \cdot \Phi(x)$$

- **Mish:**

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x))$$

Выбор функции активации зависит от задачи и архитектуры сети

Архитектура нейронной сети

Основные элементы:

1 Слои (Layers)

- Входной слой (Input layer)
- Скрытые слои (Hidden layers)
- Выходной слой (Output layer)

2 Нейроны (Neurons/Units)

- Вычислительные единицы
- Хранят веса и смещения

3 Связи (Connections)

- Полносвязные
- Сверточные
- Рекуррентные

Веса и смещения

Веса (Weights):

- Определяют силу связи между нейронами
- Обучаемые параметры
- Инициализация: Xavier, He, случайная

Смещения (Biases):

- Позволяют сдвигать функцию активации
- Увеличивают выразительность модели
- Также являются обучаемыми параметрами

Математически:

$$y = f(Wx + b)$$

где W — матрица весов, b — вектор смещений

Функция потерь (Loss Function)

Для регрессии:

- Mean Squared Error: $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- Mean Absolute Error: $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- Huber Loss

Для классификации:

- Binary Cross-Entropy: $-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
- Categorical Cross-Entropy: $-\sum_{i=1}^n \sum_{j=1}^c y_{ij} \log(\hat{y}_{ij})$
- Focal Loss

Регуляризация

Проблема переобучения:

- Модель хорошо работает на обучающих данных
- Плохо генерализует на новые данные

Методы регуляризации:

- **L1/L2 регуляризация:** штраф за большие веса
- **Dropout:** случайное отключение нейронов
- **Batch Normalization:** нормализация входов слоя
- **Early Stopping:** остановка при ухудшении на валидации

Нормализация и пакетная нормализация

Batch Normalization:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y = \gamma \hat{x} + \beta$$

Преимущества:

- Ускоряет обучение
- Позволяет использовать большие learning rate
- Уменьшает зависимость от инициализации
- Действует как регуляризатор

Альтернативы:

- Layer Normalization
- Instance Normalization
- Group Normalization

Инициализация весов

Важность правильной инициализации:

- Влияет на скорость сходимости
- Может предотвратить затухание/взрыв градиентов

Популярные методы:

- **Xavier/Glorot:** $W \sim \mathcal{N}(0, \frac{2}{n_{in} + n_{out}})$
- **He initialization:** $W \sim \mathcal{N}(0, \frac{2}{n_{in}})$
- **LeCun initialization:** $W \sim \mathcal{N}(0, \frac{1}{n_{in}})$

Правило выбора:

- Xavier — для сигмоиды, tanh
- He — для ReLU и его вариантов

Градиентный спуск

Основная идея:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta)$$

где η — learning rate, $J(\theta)$ — функция потерь

Виды градиентного спуска:

- **Batch GD:** использует весь датасет
- **Stochastic GD:** использует один пример
- **Mini-batch GD:** использует небольшую партию примеров

Проблемы:

- Выбор learning rate
- Локальные минимумы
- Седловые точки

Адаптивные методы оптимизации

AdaGrad:

$$G_t = G_{t-1} + g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

RMSprop:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

Преимущества:

- Автоматическая адаптация learning rate
- Лучше работают с разреженными данными

Adam и его варианты

Adam (Adaptive Moment Estimation):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Параметры по умолчанию: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Варианты Adam:

- AdamW (с weight decay)
- NAdam (с Nesterov momentum)
- RAdam (с rectified learning rate)

Momentum методы

Momentum:

$$v_t = \gamma v_{t-1} + \eta g_t$$

$$\theta_{t+1} = \theta_t - v_t$$

Nesterov Accelerated Gradient:

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta_t - \gamma v_{t-1})$$

$$\theta_{t+1} = \theta_t - v_t$$

Преимущества momentum:

- Ускоряет сходимость
- Помогает преодолевать локальные минимумы
- Сглаживает колебания

Learning Rate Scheduling

Стратегии изменения learning rate:

- **Step Decay:** уменьшение через фиксированные интервалы
- **Exponential Decay:** $\eta_t = \eta_0 e^{-kt}$
- **Cosine Annealing:** $\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t\pi}{T}))$
- **Warm Restart:** периодический перезапуск
- **ReduceLROnPlateau:** уменьшение при стагнации

Warm-up:

- Постепенное увеличение learning rate в начале
- Особенно важно для больших моделей

Продвинутые техники оптимизации

Gradient Clipping:

- Ограничение нормы градиента
- Предотвращает взрыв градиентов
- $g \leftarrow \frac{g}{\max(1, \frac{\|g\|}{threshold})}$

Gradient Accumulation:

- Накопление градиентов для эмуляции больших батчей
- Полезно при ограниченной памяти

Mixed Precision Training:

- Использование FP16 + FP32
- Ускорение обучения и экономия памяти

Выбор оптимизатора

Рекомендации по выбору:

- **Adam/AdamW** — хороший выбор по умолчанию
- **SGD + Momentum** — для computer vision (особенно CNN)
- **RMSprop** — для RNN и NLP задач
- **RAdam** — более стабильная версия Adam

Гиперпараметры для настройки:

- Learning rate: $[10^{-4}, 10^{-2}]$
- Batch size: степени двойки (32, 64, 128, 256)
- Weight decay: $[10^{-5}, 10^{-3}]$

Советы:

- Начинайте с малых learning rate
- Используйте learning rate finder
- Экспериментируйте с разными оптимизаторами

Современные тренды в оптимизации

Новые направления:

- **Sharpness-Aware Minimization (SAM)**
 - Поиск "плоских" минимумов
 - Улучшает генерализацию
- **Layer-wise Adaptive Rate Scaling (LARS)**
 - Для обучения с очень большими батчами
- **Automatic Mixed Precision (AMP)**
 - Автоматическое управление точностью
- **Gradient Centralization**
 - Нормализация градиентов

Процесс обучения нейронной сети

Основные этапы:

- ➊ **Forward Pass:** вычисление выхода сети
- ➋ **Loss Calculation:** вычисление функции потерь
- ➌ **Backward Pass:** вычисление градиентов
- ➍ **Parameter Update:** обновление весов

Forward Pass:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

где l — номер слоя, $g^{[l]}$ — функция активации

Проблемы при обучении глубоких сетей

Vanishing Gradient Problem:

- Градиенты экспоненциально уменьшаются в глубоких сетях
- Нижние слои обучаются медленно
- **Решения:** ReLU, ResNet, LSTM, нормализация

Exploding Gradient Problem:

- Градиенты экспоненциально увеличиваются
- Нестабильное обучение
- **Решения:** gradient clipping, правильная инициализация

Internal Covariate Shift:

- Изменение распределения входов слоев
- **Решение:** Batch Normalization

Разделение данных

Train/Validation/Test Split:

- **Training Set (60-80%):** обучение модели
- **Validation Set (10-20%):** настройка гиперпараметров
- **Test Set (10-20%):** финальная оценка

Cross-Validation:

- k-fold CV для небольших датасетов
- Более надежная оценка качества
- Вычислительно затратно для больших моделей

Стратификация:

- Сохранение пропорций классов
- Особенно важно для несбалансированных данных

Переобучение и недообучение

Переобучение (Overfitting):

- Модель слишком сложна для данных
- Хорошо на train, плохо на validation
- **Решения:** регуляризация, больше данных, early stopping

Недообучение (Underfitting):

- Модель слишком проста
- Плохо и на train, и на validation
- **Решения:** больше параметров, сложнее архитектура

Bias-Variance Tradeoff:

- Высокий bias → недообучение
- Высокая variance → переобучение
- Нужен баланс между ними

Мониторинг обучения

Метрики для отслеживания:

- Loss на train и validation
- Accuracy, Precision, Recall, F1-score
- Learning rate, градиенты, веса

Инструменты визуализации:

- TensorBoard, Weights & Biases, MLflow
- Plots: loss curves, confusion matrix, ROC-AUC

Early Stopping:

- Остановка при ухудшении на validation
- Patience: количество эпох без улучшения
- Сохранение лучших весов

Настройка гиперпараметров

Основные гиперпараметры:

- Learning rate, batch size, количество эпох
- Архитектура: количество слоев, нейронов
- Регуляризация: dropout rate, weight decay

Методы поиска:

- **Grid Search:** перебор по сетке
- **Random Search:** случайный поиск
- **Bayesian Optimization:** умный поиск
- **Hyperband, BOHB:** многоуровневые методы

Советы:

- Начинайте с грубого поиска, затем уточняйте
- Используйте логарифмическую шкалу для learning rate
- Не забывайте про валидацию

Батчи и эпохи

Терминология:

- **Эпоха:** один проход по всему датасету
- **Батч:** подмножество данных для одного обновления
- **Итерация:** одно обновление параметров

Выбор размера батча:

- Большие батчи: стабильные градиенты, быстрее
- Маленькие батчи: больше шума, лучше генерализация
- Ограничения памяти GPU
- Обычно: 32, 64, 128, 256

Количество эпох:

- Зависит от размера данных и сложности модели
- Используйте early stopping
- Мониторьте validation loss

Практические советы по обучению

Дебаггинг модели:

- Начните с переобучения на маленьком датасете
- Проверьте, что loss уменьшается
- Визуализируйте веса и активации
- Используйте gradient checking

Ускорение обучения:

- Используйте GPU/TPU
- Mixed precision training
- Gradient accumulation
- Distributed training для больших моделей

Воспроизводимость:

- Фиксируйте random seed
- Сохраняйте конфигурации
- Версионизируйте данные и код

Нейронные сети vs классическое ML

Преимущества нейронных сетей:

- Автоматическое извлечение признаков
- Работа с сырыми данными (изображения, аудио, текст)
- Высокая выразительность
- Масштабируемость с данными

Недостатки нейронных сетей:

- Требуют много данных
- Вычислительно затратны
- "Черный ящик"— плохая интерпретируемость
- Много гиперпараметров для настройки
- Склонность к переобучению

Когда использовать нейронные сети

Идеальные случаи для нейронных сетей:

- **Computer Vision:** классификация изображений, детекция объектов, сегментация
- **NLP:** машинный перевод, анализ тональности, генерация текста
- **Речь:** распознавание и синтез речи
- **Последовательности:** временные ряды, анализ ДНК
- **Большие данные:** когда есть миллионы примеров
- **Сложные паттерны:** нелинейные зависимости высокого порядка

Критерии выбора:

- Размер датасета $> 10,000$ - $100,000$ примеров
- Сложность задачи высокая
- Неструктурированные данные

Когда использовать классические ML методы

Преимущества классического ML:

- Работают с малыми данными
- Быстрое обучение и инференс
- Хорошая интерпретируемость
- Меньше гиперпараметров
- Теоретические гарантии

Лучшие случаи для классического ML:

- **Табличные данные:** банковские данные, медицинская статистика
- **Малые датасеты:** $< 10,000$ примеров
- **Интерпретируемость важна:** медицина, финансы, право
- **Быстрое прототипирование**
- **Простые паттерны:** линейные зависимости
- **Ограниченные ресурсы:** мобильные устройства, IoT

Заключение

Ключевые моменты:

- Нейронные сети — мощный инструмент для сложных задач
- Важность правильного выбора архитектуры и гиперпараметров
- Регуляризация критична для предотвращения переобучения
- Выбор между NN и ML зависит от задачи и данных

Будущее нейронных сетей:

- Еще более крупные модели (GPT-4, PaLM, Claude)
- Мультимодальность (текст + изображения + звук)
- Эффективные архитектуры (MobileNet, EfficientNet)
- AutoML и Neural Architecture Search

Спасибо за внимание!